ADR 0003 - Scenario format and eval fixtures

Status: Confirmed **Date**: 2026-06-23 **Author**: Ronin **Related**: ADR 0002, docs/BENCHMARKING.md, docs/SCORING.md, scenario.schema.json, scenario_loader.py

---

Context

BAYMAX Phase 1 needs an eval harness before building memory, voice, real APIs, RAG, fine-tuning, or multi-agent orchestration. The harness needs scenario fixtures that are:

1. Deterministic enough to run repeatedly in CI and local development
2. Structured enough for automated scoring without an LLM judge
3. Expressive enough to cover tool calls, clarification, refusal, multi-step behavior, relative time, and fake adapter state
4. Reviewable by humans during PR review
5. Independent from real Notion, Calendar, Gmail, or clipboard APIs

The scenario format is the boundary between Ronin's eval harness and Marv's agent. If this shape drifts, the scorer can no longer reliably measure whether the agent chose the right tool, supplied the right arguments, or asked for clarification when appropriate.

Decision

Phase 1 scenarios are JSON files under `scenarios/v1/`. Each file validates against `src/baymax/eval/schemas/scenario.schema.json` and loads through the Pydantic model in `src/baymax/eval/scenario_loader.py`.

Required fields:

```
id                stable scenario id, pattern <domain>_<behavior>_<nnn>
category          calendar | notion | gmail | clipboard | multi_tool
difficulty        explicit | implicit | contextual | ambiguous | multi_step
description       human-readable scenario purpose
user_input        text-only request sent to the agent
current_time      timezone-aware ISO datetime used for relative-time resolution
available_tools   exact fake tools the agent may call
initial_state     fake adapter state before execution
expected_behavior structured expected output
success_criteria  fixed vocabulary of expected checks
tags              optional labels for filtering and coverage review
```

`expected_behavior` is discriminated by `type`:

```
tool_call      exactly one expected tool call
tool_calls     ordered multi-step tool calls
clarification  no tool call; response asks for missing information
refusal        no tool call; response explains why action cannot be done
```

Audio input is explicitly out of v1. Scenario input is `user_input` text only. Relative time phrases such as "tomorrow" must be resolved through `current_time`, not the machine clock.

Reasoning

why chosen - JSON fixtures give us the simplest stable interface between agent behavior and eval scoring. They are easy to review in PRs, easy to validate with JSON Schema, and map directly into typed Python models without executable fixture logic. The explicit `available_tools` field keeps each scenario honest: the scorer can tell the difference between a valid tool call and a hallucinated unavailable tool. The `expected_behavior.type` split also prevents tool-call-only thinking; ambiguous and refusal scenarios are first-class instead of hacked around by missing tool arguments.

Alternatives considered

Alternative A - YAML scenario files

Use YAML for easier hand-writing and comments.

why rejected - comments are useful, but JSON Schema validation and Python loading are more direct with JSON. YAML also has more parsing edge cases, implicit typing surprises, and formatting inconsistency across editors.

Alternative B - Python fixture objects

Define scenarios as Python objects or factory functions in test files.

why rejected - executable fixtures are harder to inspect in code review and can accidentally depend on runtime state. Phase 1 needs static data fixtures that can be counted, validated, and converted into training data later.

Alternative C - Free-form expected behavior text

Write natural-language expected behavior and manually or semantically judge the result.

why rejected - deterministic scoring needs structured expected tools, arguments, clarification keywords, and refusal keywords. Free-form expectations would force LLM-as-judge too early and make baseline numbers less reproducible.

Alternative D - Include final-state predicates only

Score only by comparing final fake adapter state after tool execution.

why rejected - final state is useful but not enough. We also need to know whether the agent picked the correct tool, used the correct order, avoided premature tool calls, and did not hallucinate unavailable tools.

Consequences

Positive

- Scenarios are deterministic, typed, and schema-validatable
- Fake adapter state is explicit and local to each scenario
- Clarification/refusal behavior is represented directly, not as missing tool calls
- `_contains` argument checks allow realistic wording without fuzzy matching
- Scenario files can later be transformed into SFT or eval datasets

Negative

- JSON is verbose for large scenario sets
- Single-turn clarification is baked into v1; multi-turn clarification needs a new format or version
- Deterministic keyword checks can miss semantically correct alternate phrasing

Neutral

- Scenario format and Pydantic loader are duplicated sources of truth; tests must keep them aligned
- v1 fixture naming is intentionally simple and may need richer taxonomy at 200+ scenarios

---

Open questions / follow-ups

- Generate JSON Schema from Pydantic models or keep both with drift tests?
- Add state-predicate scoring once fake adapter outputs become central to task success?
- Introduce `scenarios/v2/` for multi-turn and sub-task-chain formats?
- Add scenario metadata for train/eval split when Phase 2 SFT data prep starts?

References

- `src/baymax/eval/schemas/scenario.schema.json`
- `src/baymax/eval/scenario_loader.py`
- `scenarios/v1/COVERAGE.md`
- docs/BENCHMARKING.md
- docs/SCORING.md
- ADR 0002 - Tool-call schema and contract

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.